Technical Documentation

Identification of Animals Using Artificial Intelligence

Author: Petra Oravová
Supervisor: Ing. Tomáš Goldmann, Ph.D.

May 13, 2025

1 Introduction

This document outlines the structure and functions of the muzzle detection, identification, and verification system. the system uses pre-trained deep learning models to perform image processing and comparison tasks, enabling accurate identification and verification of cows from images.

2 Project Structure

The source code and data used in this project are located in the `implementation` directory. This section describes the project structure and provides a brief overview of its components and their functionality:

- `main.py` – the entry point of the application. This script provides a command-line interface (CLI) for running various modes of the system, including detection, identification, database import, and graphical user interface (GUI).
- `requirements.txt` – List of required Python dependencies.
- `documentation.pdf` – This technical documentation.
- `detection/` – Contains code for animal detection using YOLO11n.
  - `train_detector.py` – Contains functions to train and test YOLO detection model.
  - `detect_and_crop.py` – Contains functions to run detection and crop images for further processing.
- `identification/` – Contains code for animal identification using embedding models (MobileNet, ResNet, EfficientNet).
  - `infer.py` – Contains functions for cow identification and verification.
  - `model.py` – Defines the model architecture for identification (e.g. MobileNet, ResNet).
  - `preprocess.py` – Contains a function to split the dataset.
  - `train.py` – Contains functions to train the identification model on the provided dataset.
  - `test.py` – Contains functions for testing the trained identification model on unseen data.

  - `triplet_dataset.py` – Dataset class for creating triplets used in training of the identification model.

- `data/` – Dataset folders:
  - `yolo_dataset/` – Dataset used for training detection model
  - `id_dataset_raw/` – Raw data for identification
  - `id_dataset_split/` – Cropped muzzles from id_dataset_raw split into train, val, test and newcows_test sets.
  - `pairs.json` – JSON file containing pairs of images used for fair evaluation across different networks.

- `models/` – Pre-trained model weights:
  - `mobilenet.pth` – Pre-trained MobileNet_V3_Large weight.
  - `resnet.pth` – Pre-trained ResNet-50 weight.
  - `efficientnet.pth` – Pre-trained EfficientNet_B1 weight.
  - `yolo11n_model.pt` – Pre-trained YOLO11n weight.

- `db/` – Database utilities:
  - `connection.py` – Contains functions for managing database connections.
  - `utils.py` – Utility functions for database operations.
  - `backup.py` – Contains functions for creating and restoring database backups.
  - `backups/` – Contains database backup files such as `mobilenet.dump`, `resnet.dump`, `efficientnet.dump`.

- `app/` – Code for the graphical user interface (GUI) of the application, implemented using PyQt5.
  - `gui.py` – Main script for the GUI interface that interacts with the user for tasks such as cow identification and verification.

3 Function Descriptions

This section describes all functions implemented in the source code. Each subsection corresponds to a source code directory and provides a summary of the purpose and parameters of the functions contained within.

3.1 detection/

`train_detector.py`

- `train_detector()`
  Trains a YOLO11n object detector using a pre-trained model. Saves the trained weights to the default output directory (`runs/detect/train`).

- `test_detector(model_name="models/yolo11n_model.pt")`
  Tests a trained YOLO11n model on the test dataset. the model path can be optionally provided; otherwise, a default model path is used.

`detect_and_crop.py`

- `detect_and_crop(input_path, resize_dim=(256, 256))`
  Loads an image, detects muzzle using the YOLO model, crops the most confident detection, and resizes it to the specified dimensions. Returns the cropped image and the bounding box coordinates.

- `process_folder(input_folder="data/id_dataset_raw", output_folder="data/id_dataset_crop")`
  Applies detection and cropping to all images in the given input folder. Saves the cropped results to the output folder while preserving the directory structure.

3.2 identification/

`model.py`

- `EmbeddingModel(nn.Module)`
  Base class for all embedding models. Defines the interface and shared logic for generating L2-normalized 256-dimensional embeddings from images. Implements:
  - `forward(x)` – Passes input image through the base network and a projection layer.
  - `get_embedding(image_path, device)` – Loads and preprocesses an image, then returns its normalized embedding.

- `EfficientNetB1Embedding(EmbeddingModel)`
  Subclass implementing an EfficientNet-B1 based embedding model. Removes classifier and projects features from 1280 to 256 dimensions.

- `ResNetEmbedding(EmbeddingModel)`
  Subclass implementing a ResNet-50 based embedding model. Removes classifier and projects features from 2048 to 256 dimensions.

- `MobileNetEmbedding(EmbeddingModel)`
  Subclass implementing a MobileNet_V3_Large based embedding model. Removes classifier and projects features from 960 to 256 dimensions.

- `MODEL_REGISTRY`
  Dictionary that maps model names (`"efficientnet"`, `"resnet"`, `"mobilenet"`) to their class constructors.

`triplet_dataset.py`

- `TripletDataset(Dataset)`
  PyTorch dataset class for generating image triplets used in metric learning (e.g., training with triplet loss). Supports both dynamic triplet generation (for training) and static precomputed triplets (for testing).

  - `__init__(root_dir, transform=None, is_training=True)` – Initializes the dataset and optionally precomputes triplets depending on the mode (training/evaluation).
  - `__len__()` – Returns the number of samples (depends on whether the dataset is in training or testing mode).
  - `load_images(anchor_path, positive_path, negative_path)` – Loads and applies transformations to a triplet of images.
  - `create_triplets()` – Precomputes and returns a list of reproducible image triplets for testing.
  - `create_triplet()` – Dynamically generates a new image triplet for training.
  - `__getitem__(idx)` – Returns either a precomputed triplet or a dynamically generated one, depending on mode.

`preprocess.py`

- `split_dataset(dataset_folder="data/id_dataset_crop", output_folder="data/id_dataset_split")`
  Splits the dataset into training, validation, and testing subsets. Also, some identities are reserved for "new cow" testing scenarios. the input dataset should have subfolders for each class (cow ID).

`train.py`

- `train_epoch(model, train_loader, optimizer, criterion, device)`
  Trains the given model on a single epoch using triplet loss. Returns the average loss value.

- `validate_epoch(model, val_loader, criterion, device)`
  Evaluates the model performance on the validation set for one epoch. Returns the average validation loss.

- `train_model(model_name="mobilenet", output_name="models/best_model.pth")`
  Initializes the model, trains it over multiple epochs, evaluates on validation data, and saves the best model weights based on validation loss.

`test.py`

- `plot_genuine_impostor_histogram(distances, model, save_path=None)`
  Visualizes the distribution of distances between genuine and impostor image pairs as a histogram. Optionally saves the plot as a PDF. Useful for analyzing model separability.

- `generate_and_save_pairs(save_path="data/pairs.json", max_same_pairs_per_cow=15)`
  Generates evaluation image pairs (same and different cow IDs) from the `newcows_test` dataset and saves them to a JSON file for reproducible testing.

- `evaluate_model_on_dataset(model, test_folder, threshold, mode="identification")`
  Evaluates a trained model on a test dataset in either `identification` or `new_cows` mode. Returns a dictionary of statistics (e.g., true positives, false positives).

- `compute_metrics(stats, threshold)`
  Computes key evaluation metrics such as accuracy, precision, and recall from the collected evaluation statistics.

- `run_experiment(model_name="mobilenet", model_path="models/mobilenet.pth", mode="identification", thresholds=None)`

Runs a full evaluation experiment for a given model and set of thresholds. Supports both identification and verification (new_cows) modes.

`infer.py`

- `infer(image_path, model_name="mobilenet", model_path="models/mobilenet.pth", threshold=0.5)`
  Identifies the cow in a given image by detecting the muzzle, extracting an embedding, and comparing it against the known database. Returns the matched cow ID and the confidence distance, or a message if detection fails, along with the detected bounding box.

- `verify(image1_path, image2_path, model_name="mobilenet", model_path="models/mobilenet.pth", threshold=0.5)`
  Compares two images to determine whether they belong to the same cow using embedding similarity. Returns a boolean indicating the match, the computed distance, and bounding boxes for both images.

3.3 db/

`connection.py`

- `get_connection()`
  Establishes and returns a connection to the PostgreSQL database using `psycopg2`.

- `close_connection(conn)`
  Closes an active PostgreSQL database connection.

- `execute_query(query, params=None, fetchone=False, fetchall=False)`
  Executes a SQL query on the PostgreSQL database. Supports optional parameters and result fetching (either a single row or all rows).

- `create_database()`
  Verifies the existence of the database and creates it if it does not exist.

- `create_table()`
  Creates the `cow_embeddings` table in the database and ensures that the vector extension is enabled for embedding storage.

`utils.py`

- `insert_cow(cow_id, embedding)`
  Inserts a cow's ID and its corresponding embedding vector into the `cow_embeddings` table in the database.

- `identify_cow(embedding, threshold=0.3)`
  Identifies the closest matching cow in the database based on the provided embedding. Returns the cow ID and distance if the match is found within the given threshold.

- `cow_exists(cow_id)`
  Checks whether a cow with the specified ID already exists in the `cow_embeddings` table.

- `fill(model_name="mobilenet", model_path="models/mobilenet.pth")`
  Populates the database with cow embeddings by generating embeddings for each image in the training dataset using the specified model.

`backup.py`

- `export_database(output_filename)`
  Exports the PostgreSQL database to a backup file. the backup is saved with the given `output_filename` to db/backups/.

- `import_database(input_filename)`
  Imports the PostgreSQL database from a backup file in db/backups/ specified by `input_filename`.

3.4 app/

`gui.py`

- `SettingsDialog(QDialog)`
  a dialog for configuring model settings, including model selection, threshold, and distance display options.
  - `__init__(self, parent=None)`: Initializes the dialog with the model combo box, threshold input, and distance checkbox.
  - `on_model_changed(self)`: Updates the database based on the selected model.

- `_load_pixmap_from_pil(pil_image)`
  Converts a PIL image to QPixmap for GUI display.

- `_load_pixmap(path)`
  Loads an image from a file and converts it to a QPixmap for GUI display.

- `CowApp(QWidget)`
  Main application window for cow identification and verification.

  - `__init__(self)`: Initializes the main window, sets up the layout, and adds functionality for loading images, displaying results, and handling settings.
  - `resizeEvent(self, event)`: Resizes tabs dynamically based on window size.
  - `update_tab_widths(self)`: Dynamically adjusts the width of each tab to fill available space.
  - `build_identify_tab(self)`: Constructs the "Identify" tab of the GUI, allowing users to load image and perform identification.
  - `build_verify_tab(self)`: Constructs the "Verify" tab of the GUI allowing users to load images and perform verification.
  - `load_image1(self)`: Loads the first image from disk and updates the corresponding preview.
  - `load_image2(self)`: Loads the second image for verification and updates the preview.
  - `run_identify(self)`: Runs cow identification on the loaded image and updates the UI with results.
  - `run_verify(self)`: Runs cow verification between two loaded images and updates the UI with results.

3.5 `main.py`

- `main()` The main entry point of the application, used to run various operation modes via command-line argument `--mode`.

  - Creates an argument parser and processes options for tasks, such as preprocessing, training, inference, and launching the GUI.
  - Dispatches control to the appropriate function based on the selected mode.

- `MODES`
  Dictionary mapping mode names (e.g., `test_prepare`, `id_train`, `app`) to their corresponding handler functions and descriptions. Acts as a centralized registry for all available CLI functionalities, enabling dynamic selection of operations.

- `run_test_prepare(args)`
  Generates and saves image pairs for identification model testing.

- `run_preprocess(args)`
  Performs dataset preprocessing and splits it into training and validation sets.

- `run_id_train(args)`
  Trains the identification model with an optional specification of architecture (`--model`) and output name (`--output`).

- `run_detect_train(args)`
  Trains an object detection model to locate muzzle in input images.

- `run_detect_test(args)`
  Tests the trained object detection model, optionally using a custom model path (`--model_path`).

- `run_crop(args)`
  Detects and crops animals from images in a given folder using the trained detector.

- `run_fill(args)`
  Fills the database with embeddings using the specified model architecture (`--model`) and weights (`--model_path`).

- `run_export(args)`
  Exports the internal database to a file specified by `--output`.

- `run_import(args)`
  Imports a database from a file specified by `--input`.

- `run_test(args)`
  Runs evaluation of the identification or verification model. Allows optional model architecture (`--model`), weights path (`--model_path`), and test mode (`--settings`).

- `run_infer(args)`
  Performs inference on a single image. Requires the image path (`--image`); model architecture (`--model`) and weights (`--model_path`) can be optionally specified.

- `run_app(args)`
  Launches the Graphical User Interface (GUI) using PyQt5.

4 Additional Notes

- Model weights and database dumps are included to allow reproducibility without the need for retraining.

- For training and testing the YOLO detector, absolute paths must be specified in the `data.yaml` configuration file.

- the project is modular by design, making it easy to extend or replace components such as detection or embedding models.

- System requirements, initial database setup, and usage examples are described in `README.md`.

- List of all supported modes and arguments can be viewed by running `python main.py --help`.

- For a detailed explanation of the system design, model training, evaluation methods, and results, please refer to the accompanying bachelor's thesis (xoravo01-Identifikace-zvere-pomoci-AI.pdf).